Ingegneria del Software
Prova pratica AA 2025-26 – Progetto Python
Classificazione binaria con feature reduction QUBO
Durata: 2 settimane dalla data della consegna[1]

Descrizione sintetica del progetto

Il progetto riguarda lo sviluppo di un'applicazione per classificare dei dati contenuti in un dataset, chiamati "campioni" o "record". Si tratta di una classificazione binaria, cioé dell'assegnazione di un campione a una classe a due valori, come ad es. "sano"/"malato", "rischio basso/rischio alto", "amico/nemico".
Il dato da classificare è un record, composto da una lista di campi, che nel nostro caso sono numerici.
Il dataset iniziale ha i campioni già classificati, cioè per ogni campione si conosce la sua classe. Esso va diviso in due sottoinsiemi. Uno, chiamato "training set", serve all'addestramento del classificatore; l'altro, chiamato "test set" serve a valutare la qualità del classificatore addestrato sul training set.
Si può considerare il dataset come una matrice, le cui righe sono i campioni, e le cui colonne sono i campi, chiamati anche "features" (caratteristiche); i valori noti di ogni campioni sono contenuti in un vettore binario (di 0 e 1), il cui elemento i-esimo rappresenta la classe del campione i-esimo.

Più in dettaglio, la classificazione proposta riguarda la determinazione se una persona che richiede un prestito sia affidabile o a rischio. Il programma sviluppato deve però semplicemente gestire un dataset numerico dato, con colonna target di zeri e uni.
Ogni richiesta di prestito è caratterizzata da un dato numero di campi (chiamati "features") che descrivono il richiedente, con dati anagrafici, lavorativi, finanziari, e altri. L'output è un valore binario: "0" per richiedente affidabile, "1" per richiedente a rischio.
Se i campi sono troppi (ad es., il dataset usato per le prove ha 145 features), prima di predisporre il classificatore occorre ridurre il numero di features, con una procedura di ottimizzazione.
In sintesi, le operazioni da svolgere, che saranno descritte in dettaglio in seguito, sono:

1. Lettura del dataset per la riduzione delle features e l'apprendimento.
2. Conversione di tutti i campi dei record a valori numerici normalizzati. Eliminazione preliminare delle features (colonne) che contengono una percentuale di valori nulli o non definiti superiore a una data soglia (di solito intorno all'90-95%).
3. Divisione del dataset in training set e test set. È effettuata considerando i primi M campioni come training set, e i campioni restanti come test set. M è fornito in input al programma.
4. Determinazione di quali features eliminare e quali usare per la classificazione, usando i dati del training set e con una procedura di ottimizzazione QUBO.
5. Eliminazione delle features non selezionate sia dal training set che dal test set.
6. Addestramento di un classificatore sui record del training dataset così ridotto.
7. Classificazione sui dati del test set, confronto coi valori reali e stampa dei risultati.

L'applicazione dovrà essere dotata di un'interfaccia utente per controllare e visualizzare lo stato delle varie fasi dell'elaborazione, e di test automatici per la verifica del sistema
Gli algoritmi ed approcci non sono determinati, tranne la procedura per calcolare la funzione costo per la riduzione delle features.

1 In caso di problemi, si può richiedere un'estensione al docente.

Il programma sviluppato deve poter elaborare dataset di dimensioni qualsiasi (sia righe che colonne). L'unico vincolo è che i dati siano già in forma numerica. Il vettore target è contenuto in una colonna del dataset il cui nome è fornito in input.
Il progetto deve essere sviluppato da un gruppo formato da una coppia da due studenti, utilizzando un LLM, e consegnato tramite repository GitHub. In casi particolari, potrà essere autorizzato un "gruppo" formato da un solo studente; in tale caso, il progetto rimane comunque invariato.

2. Vincoli generali

Il progetto deve essere sviluppato in **Python 3.11 o superiore**.

Vanno usate librerie open source, scelte dal gruppo anche con l'ausilio del LLM.

È obbligatorio usare almeno lo strumento LLM/AI prescritto per il gruppo. L'interazione deve avvenire in lingua inglese. L'uso deve essere documentato in grande dettaglio.

3. Repository GitHub

Ogni gruppo deve creare un repository GitHub con nome:

`isw-qubo-classification-GXX`

dove `GXX` è il codice del gruppo, ad esempio:

```
python-qubo-classification-G01
python-qubo-classification-G02
```

Il repository deve essere accessibile agli utenti Github: micheledmi, Druimo e Piras78.

4. Struttura obbligatoria del repository

Il repository deve avere esattamente questa struttura minima:

```
isw-qubo-classification-GXX/
│
├── README.md
├── requirements.txt
├── group_info.yaml
│
├── data/
│   └── input_dataset.csv
│   └── sample_test_dataset.csv
│
├── src/
│   └── qubo_project/
│       ├── __init__.py
│       ├── preprocessing.py
│       ├── feature_selection.py
│       ├── model.py
│       └── gui.py                # opzionaleo simile per la GUI
│
├── tests/         # questa cartella contiene i test automatici
```

```
│
├── outputs/
│
└── reports/
    ├── project_report.yaml
    └── llm_usage/
        ├── LOG-GXX-01.md
        ├── LOG-GXX-02.md
        ├── ...
        └── files/
            ├── LLM_GXX-MM-DD-001_preprocessing_function_1.py
            ├── LLM_GXX-MM-DD-001_qubo_creation_1.py
            ├── LLM_GXX-MM-DD-001_final_report_draft.yml
            └── . . .
```

La cartella `outputs/` può essere vuota alla consegna. Sarà usata dal programma di valutazione per salvare risultati, modelli e predizioni.

5. File `group_info.yaml`

Il file `group_info.yaml` deve essere in formato YAML e contenere le informazioni del gruppo. Se il gruppo non conosce yaml, può scrivere le informazioni in formato JSON e usare un convertitore, o farsi aiutare dal LLM.

L'ultimo campo del file è una flag che dice se il progetto è terminato.

Esempio:

```yaml
group_id: G01
students:
  - matricola: "123456"
  - matricola: "654321"
main_llm_used: "ChatGPT GPT-5"
paid_subscription_used: true
second_llm_used: "Qwen Studio Qwen3.6-Plus"
third_llm_used: ""
notes: "Il gruppo ha usato ChatGPT principalmente per codifica, debugging e spiegazione del modello QUBO. Ha usato Qwen Studio per la generazione della GUI"
project_finished: "false"    # Mettere "true" se consegnato
```

6. Dataset

Il dataset viene fornito al sistema in formato CSV.

Il dataset contiene:

- N colonne di input con campi numerici;
- una ulteriore colonna target per la classificazione binaria;

- alcune colonne possono essere vuote o quasi vuote, o contenere tutti zeri, o quasi.

Il nome della colonna target sarà passato al programma tramite parametro di input.

Il codice non deve assumere che il target abbia sempre lo stesso nome.

Il codice deve supportare target binario codificato come: 0 / 1

Sarà fornito un dataset, di nome: `input_dataset.csv`, per effettuare le prove, con N=145 features più il target, e 20.000 campioni. Questo non è il dataset che sarà usato per la verifica del programma sviluppato.

Il dataset di verifica avrà più di 1.500.000 campioni, quindi i programmi sviluppati devono poter gestire dataset di tali dimensioni.

7. Fase 1: preprocessing del dataset

Il modulo di preprocessing deve:

1. leggere il CSV con tutto il dataset di input;
2. separare la colonna target;
3. eliminare colonne vuote o quasi vuote, ovvero con tutti zeri o quasi; la percentuale di colonne vuote o con zeri sul totale sotto la quale la colonna è eliminata è un parametro dato in input prima dell'elaborazione;
4. normalizzare le feature, producendo un dataset finale numerico e normalizzato;

7.1 Colonne vuote o quasi vuote

Una colonna può essere eliminata se contiene troppi valori mancanti o pari a zero.

La soglia di rimozione è data in input al programma. Ad esempio, potranno essere rimosse colonne con più del 95% di valori mancanti o nulli.

7.2 Normalizzazione

Le feature devono essere normalizzate usando la standardizzazione z-score, cioè i valori sono trasformati con una trasformazione lineare in modo da avere media nulla e deviazione standard unitaria.

La normalizzazione deve essere applicata sull'intero dataset, escluso ovviamente la colonna col target.

7.3 Divisione del dataset

Il dataset è suddiviso considerando i primi M campioni come training set, e i campioni restanti come test set. Il parametro M va calcolato in base alla percentuale del test set sul totale dei campioni, fornita in input. La divisione del dataset va fatta con un "taglio netto" al campione M.

Entrambi i dataset (training e test) sono salvati su file .csv.

8. Fase 2: feature reduction tramite QUBO

La riduzione delle feature deve essere formulata come problema QUBO.

Dato un dataset normalizzato con m feature, il sistema deve selezionare una percentuale di feature data in input al programma, più o meno una tolleranza data

Ad esempio, se m è il numero di feature disponibili dopo il preprocessing, e la percentuale di feature voluta è il 20% ±1 feature, il numero atteso di feature selezionate, K, sarà:

K = round(0.20 * m) ±1.

8.1 Vettore delle feature selezionate

L'output della fase QUBO deve essere un vettore binario:

x* = [0, 1, 0, 1, 0, ...]

dove:

- 1 significa feature selezionata;
- 0 significa feature non selezionata.

La lunghezza del vettore deve essere uguale al numero di feature disponibili dopo il preprocessing (K), esclusa la colonna target.

8.2 Funzione costo QUBO

La funzione costo deve essere calcolata secondo le formule date nelle slide relative, e riportate qui nell'Appendice. Essa varia in funzione di un parametro α che è compreso tra 0 e 1. Al variare del parametro α cambia il numero di feature selezionate. In particolare, per α = 0 non è selezionata alcuna feature, per α = 1 sono selezionate tutte.

Occorre quindi variare opportunamente α sino a trovare un valore tale che l'ottimizzazione restituisca un vettore QUBO x* contenente un numero K di "1", pari alla percentuale di feature voluta, più o meno la tolleranza.

8.3 Algoritmo di ottimizzazione

L'algoritmo di ottimizzazione QUBO è scelto dal gruppo. Potrà essere codificato dal gruppo, oppure selezionato in una delle librerie open source disponibili, anche su indicazione del LLM.

La scelta dell'algoritmo deve essere documentata.

8.4 Riduzione delle feature di training

Il vettore binario ottimo risultante dall'algoritmo di ottimizzazione è usato per ridurre le feature, producendo un training set ridotto con K colonne, più la colonna target.

Tale dataset deve essere memorizzato in un file .csv, inclusa la colonna coi dati di target.

9. Fase 3: learning del classificatore

Il gruppo deve implementare tre algoritmi di classificazione binaria, di cui uno deve essere Random Forest. Altri, ad esempio, possono essere:

- logistic regression;
- decision tree;

- support vector machine;
- k-nearest neighbors;
- gradient boosting;
- altri classificatori disponibili in librerie open source.

Si deve poi dare in input al programma il nome un classificatore tra i tre, e questo deve essere addestrato usando solo le feature selezionate tramite il vettore x* trovato nella fase QUBO.

Il modello addestrato deve poter essere salvato su file, in modo che il programma di valutazione possa riutilizzarlo. Il formato consigliato è .joblib. Occorre anche salvare statistiche relative al training, tra cui il tempo di training.

10. Fase 4: classificazione dei dati di test

Il classificatore addestrato deve essere poi applicato al dataset di test, composto dai dati di test estratti dal dataset originale (vedi punto 7.3). Il sistema deve:

1. leggere il dataset di test;
2. selezionare le stesse K feature scelte nella fase QUBO;
3. salvare su file .csv il dataset di test ridotto, inclusa la colonna coi dati di target.
4. caricare il classificatore addestrato;
5. produrre le predizioni.

Va prodotto un file .csv di output che deve contenere la classificazione di tutti i campioni, con i dati:

```
row_n,target,prediction,score
0,1,1,0.83
1,0,1,0.21
2,1,1,0.77
```

Dove:

- `row_n` è l'indice della riga
- `target` è la classe reale, fornita col dataset: `0` o `1`;
- `prediction` è la classe predetta dal classificatore: `0` o `1`;
- `score` è il punteggio o probabilità della classe positiva.

Oltre all'output per ogni record del dataset di test, il programma deve anche visualizzare e salvare su file le seguenti statistiche sulla bontà della classificazione (vedi alla sez. 11.3 per il formato):

- Nr. totale campioni
- Percentuale di campioni con target = 1
- Precision, Recall e F1 dei campioni con target = 0
- Precision, Recall e F1 dei campioni con target = 1
- ROC-AUC
- Matrice di confusione

11. Interfacce Python obbligatorie

Il programma di valutazione automatico userà sia la linea di comando sia le funzioni Python del progetto. Per questo motivo i moduli devono rispettare i nomi e le interfacce seguenti.

11.1 `preprocessing.py`

File obbligatorio:

`src/qubo_project/preprocessing.py`

Funzione obbligatoria:

```
def fit_normalize(
    input_csv: str,                 # Input dataset name
    target_column: str,             # column name of target
    normalized_csv: str,            # Name of output normalized data set
    outInitalRes_json: str,         # Name of output statistics and data file
    minPercValid: float = 0.05,  # Minimum % of valid non-zero data for a column
):
    ...
```

La funzione deve:

- leggere il dataset fornito, con la prima riga che contiene i nomi delle colonne, e che ha la colonna target di nome `target_colum`;
- eliminare le feature con pochi campi, cioé con meno del `minPercValid` percento di feature non zero o non presenti;
- normalizzare il dataset, esclusa la colonna target;
- salvare il dataset numerico normalizzato in `normalized_csv`, mantenendo nella prima riga l'intestazione originale delle colonne, inclusa la colonna coi valori di target;
- salvare i dati sul preprocessing in formato .json

Il file JSON di output deve avere questa struttura (i dati sono solo di esempio):

```
{
  "n_input_features": 140,
  "n_kept_features": 120,
  "dataset_size": 20000
  "dataset_input_time": 2.34,
  "dataset_processing_time": 3.02,
  "dropped_feature_names": ["feature_1", "feature_20"]
}
```

11.2 `feature_selection.py`

File obbligatorio:

`src/qubo_project/feature_selection.py`

Funzione obbligatoria:

```
def select_features(
```

```
    normalized_csv: str,   # Input dataset name
    reducedTrain_csv: str, # Name of output training dataset with reduced feat.
    reducedTest_csv: str,  # Name of output test dataset with reduced features
    output_ottim_csv: str, # Name of output optimization data varying alpha
    output_json: str,      # Name of output statistics and data file
    target_column: str,    # Column name of target
    percTest: float = 0.30 # % of test data with respect to the dataset size
    percSelected: float = 0.20,  # percentage of features to select
    allowance: int = 1,    # Allowance of features to select
    seed: int = 42,        # Seed for random repeatibility
    alpha_computations: int = 100   # Max. n. of optimizations varying alpha
):

    ...
```

La funzione deve:

- leggere il dataset normalizzato dal file `normalized_csv`, con la prima riga che contiene i nomi delle colonne, e che ha la colonna target di nome `target_column`;
- costruire il problema QUBO, in funzione del parametro α;
- eseguire l'algoritmo di ottimizzazione variando α in modo da selezionare K feature, con K pari alla percentuale data `percSelected` di feature, più o meno un numero di feature pari alla tolleranza `allowance`; ATTENZIONE! Il problema QUBO **non** deve prevedere un numero esatto di feature K, ma variare α fino a trovare il numero voluto, più o meno la tolleranza. Per evitare cicli infiniti, è dato il numero massimo di calcoli variando il parametro α.
- salvare il vettore binario finale, i nomi delle feature selezionate, il tempo di calcolo della matrice Q e la media e dev. standard dei tempi di ottimizzazione variando α in `output_json`;
- salvare i dati di tutte le ottimizzazioni fatte variando α nel file di nome `output_ottim_csv.csv`, le cui righe contengono: valore di α in ordine crescente, tempo di ottimizzazione, numero di feature (numero di "1" del vettore ottimo), valore della funzione costo trovato.
- eliminare le feature non selezionate dal dataset di input, dividerlo in training set e test set, e salvarli in `reducedTrain_csv` e `reducedTest_csv`, mantenendo nella prima riga l'intestazione originale delle colonne selezionate, e nell'ultima colonna i valori target.
- Il problema QUBO si costruisce usando una funzione costo che massimizza la somma delle correlazioni di Spearman tra feature e vettor target (influenza), e minimizza le correlazioni tra feature differenti (indipendenza). Le correlazioni sono sempre il modulo delle correlazioni. Il fattore influenza è pesato dal parametro α, l'indipendenza da (1 – α). α è compreso tra zero e uno.

Il file JSON di output deve avere questa struttura (i dati sono solo di esempio):

```
{
  "n_features": 95,
  "target_ratio": 0.20,
  "target_k": 19,
  "allowance": 1,
  "n_selected": 19,
  "alpha": 0.344,
```

```
  "selected_vector": [1, 0, 0, 1, ...., 0, 0],
  "selected_feature_names": ["feature_1", "feature_4"],
  "algorithm": "simulated_annealing",
  "seed": 42,
  "alpha_computations": 6,
  "percTest": 0.30,
  "training_dataset_size": 14000,
  "test_dataset_size": 6000,
  "q_matrix_creation_time": 2.53,
  "mean_optimization_time": 0.23,
  "std_dev_optimization_time": 0.044,
}
```

11.3 `model.py`

File obbligatorio:

`src/qubo_project/model.py`

Funzione obbligatoria per il training:

```
def train(
    classifier: str,       # classifier to use
    reducedTrain_csv: str, # training dataset
    target_column: str,    # target column name
    model_path: str,       # saved trained classifier
    metrics_json: str,     # file with training statistics
    seed: int = 42,
):
    ...
```

La funzione deve:

- selezionare tra i tre classificatori disponibili quello indicato.
- leggere il dataset di training normalizzato ridotto;
- addestrare il classificatore;
- salvare il modello in `model_path`;
- salvare metriche di training o validation in `metrics_json`;

Il file JSON di output deve avere questa struttura (i dati sono solo di esempio):

```
{
  "classifier": "random_forest",
  "seed": 42,
  "training_dataset": "training_reduced.csv",
  "target_column": "target",
  "model_path": "model80.joblib",
  "n_samples": 14000,
  "n_features": 19,
  "target_1_percentage": 1.59,
  "dataset_input_time": 0.16,
  "training_time": 1.61
}
```

Funzione obbligatoria per la predizione:

```
def predict(
    reduced_Test_csv: str,      # Input test set
    target_column: str,         # Target column name
    model_path: str,            # saved trained classifier to use
    predictions_csv: str,       # Output predictions
    classif_stats_json: str,    # File with classification stats
):
```

La funzione deve:

- leggere il dataset ridotto di test;
- caricare il classificatore salvato su file;
- generare le predizioni;
- salvare nel file `predictions_csv` i risultati per ogni record del dataset di test, secondo quanto indicato nella sezione 10.
- salvare nel file `classif_stats_json` le statistiche di qualità della classificazione.

Il file JSON di output deve avere questa struttura (i dati sono solo di esempio):

```
{
  "classifier": "random_forest",{
  "n_samples": 6000,
  "target_1_count": 375,
  "target_1_percentage": 1.56,
  "accuracy": 0.9914166666666666,
  "class_0": {
    "precision": 0.9926789245592629,
    "recall": 0.9986455026455027,
    "f1": 0.9956532748143146,
    "support": 23625
  },
  "class_1": {
    "precision": 0.8626609442060086,
    "recall": 0.536,
    "f1": 0.6611842105263158,
    "support": 375
  },
},
  "roc_auc": 0.9663519153439154,
  "confusion_matrix": {
    "labels": [
      0,
      1
    ],
    "matrix": [
      [
        23593,
        32
      ],
```

```
      [
        174,
        201
      ]
    ]
  }
}
```

12. Interfaccia da linea di comando obbligatoria

I file contenenti le funzioni sopra indicate devono anche essere eseguibili, fornendo i parametri necessari via riga di comando, secondo il formato indicato di seguito (i dati forniti sono di esempio).

File `preprocessing.py`:

```
>python preprocessing.py \
  --input dati_credito.csv \
  --target target \
  --out-data normalized.csv \
  --out-json preprocessing_result.json \
  --min-perc-valid 0.06
```

Output atteso:

```
outputs/normalized.csv
outputs/preprocessing_result.json
```

File `feature_selection.py`:

```
>python feature_selection.py \
  --in-normalized normalized.csv \
  --out-train training_reduced.csv \
  --out-test test_reduced.csv \
  --out-optimizations optimizations.csv \
  --out-json feature_selection_result.json \
  --target target \
  --perc-selected 0.20 \
  --allowance 1 \
  --perc-test 0.30 \
  --seed 42 \
  --alpha-computations 10
```

Output atteso:

```
outputs/training_reduced.csv
outputs/test_reduced.csv
outputs/optimizations.csv
outputs/feature_selection_result.json
```

File `model.py`:

Per lanciare il training:

```
>python model.py train \
  --classifier random_forest \
```

```
  --in-reduced training_reduced.csv \
  --target target \
  --out-model model.joblib \
  --out-metrics training_metrics.json \
  --seed 42
```

Output atteso:

```
outputs/model.joblib
outputs/training_metrics.json
```

Per lanciare la classificazione (su file di test, ma anche su qualunque altro file “ridotto”, cioè solo con le feature selezionate):

```
>python model.py predict \
  --input-testset test_reduced.csv \
  --target target \
  --model model.joblib \
  --out-predictions predictions.csv \
  --out-stats classification_stats.json
```

Output atteso:

```
outputs/predictions.csv
outputs/classification_stats.json
```

13. Test automatici

Il repository deve contenere test automatici eseguibili con:

```
pytest
```

I test devono verificare almeno:

1. che il preprocessing produca solo colonne numeriche;
2. che il preprocessing gestisca valori mancanti;
3. che la normalizzazione produca un dataset valido;
4. che la feature selection produca un vettore binario;
5. che il numero di feature selezionate sia circa il 20%;
6. che il training produca un modello salvato;
7. che la predizione produca un file CSV con le colonne richieste.

I test devono usare un piccolo dataset di esempio creato dal gruppo e contenuto in:

```
data/sample_test_dataset.csv
```

Il dataset di test può essere estratto dal dataset originale come sottoinsieme di esso, curando che siano presenti record con entrambi i valori del target. non meno del 10%). In questa stessa cartella sarà inserito in fase di valutazione un nuovo dataset per verificare la validità del progetto su nuovi dati. Alla consegna, eliminare tutti i dataset di prova nella cartella `data/`, escluso il dataset per i test automatici.

14. GUI

Il gruppo deve realizzare una semplice GUI, con un controllo basico di possibili input errati o dati non conformi, per permettere a un utente umano di:

1. selezionare un dataset;
2. avviare il preprocessing;
3. avviare la feature selection;
4. avviare il training;
5. eseguire predizioni;
6. visualizzare o salvare gli output principali.

La GUI può essere realizzata con qualsiasi tecnologia Python.

Esempi:

- `tkinter`;
- `streamlit`;
- `gradio`;
- `PyQt`;
- altra libreria scelta dalla coppia.

La GUI deve essere avviabile con un comando documentato nel `README.md`, ad esempio:

```
python -m qubo_project.gui
```

oppure:

```
streamlit run src/qubo_project/gui.py
```

La GUI sarà valutata manualmente, non dal programma automatico.

15. Report finale

Il file in formato YAML con il rapporto finale del gruppo va scritto nel file:

```
reports/project_report.yaml
```

Esso deve contenere le informazioni desunte direttamente dal “template” allegato: `project_report_template.yaml`

Il report va compilato in lingua inglese corretta. Se ciò fosse un problema, è accettabile anche l’italiano. La completezza e accuratezza della compilazione del report finale avranno impatto sulla valutazione del progetto.

16. Log di tutte le interazione con LLM

Tutte le interazioni del gruppo con l’LLM o gli LLM usati devono avvenire in lingua inglese.

Tali interazioni devono essere documentate in un insieme di file in formato “Markdown” (.md). È stato predisposto ed è allegato un “template” (`llm_interaction_log_template.md`), che va duplicato tante volte quanto serve, dandogli il nome prescritto: “`LOG-GXX-NN.md`”, ove `GXX` è il codice del gruppo e `NN` è un numero progressivo.

Nella sottocartella “file” vanno inseriti tutti i file scaricati o comunque generati da LLM. Nel caso i programmi Python (non semplicemente gli spezzoni di codice) siano forniti dall’LLM nella risposta testuale al prompt, tali programmi vanno anche trasferiti su file .py e salvati nella sottocartella, con un nome univoco.

Le informazioni da riportare sono descritte direttamente nel “template” stesso.

La completezza e accuratezza delle registrazioni con LLM avranno impatto sulla valutazione del progetto.

17. Requisiti di riproducibilità

Il progetto deve essere riproducibile. Tutto il progetto su Github sarà scaricato come file .zip, riprodotto in locale ed eseguito.

Per questo motivo:

1. tutte le dipendenze devono essere indicate in `requirements.txt`;
2. non devono esserci path assoluti nel codice;
3. il codice deve funzionare da terminale;
4. il codice non deve richiedere input manuale durante la valutazione automatica;
5. il codice deve usare un parametro `seed` quando possibile;
6. il codice non deve scaricare dati da Internet durante la valutazione;
7. il codice non deve richiedere credenziali personali;
8. il codice non deve dipendere dalla GUI per eseguire la riga di comando;
9. prima della consegna, controllare che sia possibile scaricare tutto il progetto da Github come file .zip. Se il progetto è troppo grande, caricare su un utenza Drive ad accesso libero lo zip del progetto, e inviare i link a docente e tutor con una mail.

18. Errori da evitare

Saranno penalizzati:

1. repository con struttura diversa da quella richiesta;
2. assenza di `requirements.txt`;
3. codice eseguibile solo da notebook;
4. path assoluti nel codice;
5. assenza delle funzioni obbligatorie;
6. assenza della CLI standard;
7. preprocessing applicato in modo diverso a training e test;
8. uso dei dati di test per addestrare scaler, encoder o modello;
9. feature selection non basata su QUBO;
10. numero di feature selezionate molto diverso dal parametro dato;
11. assenza di modello salvato;
12. assenza di test automatici;
13. mancata documentazione dell’uso dell’LLM;
14. output non nei formati richiesti.

Si ricorda che la valutazione automatica sarà eseguita esplicitamente sui file di output nel formato richiesto. Questi file vanno creati a tempo di runtime, in quanto dovrà essere possibile eseguire il programma anche su dataset esterni.

19. Consegna

La consegna consiste nel repository GitHub completo, strutturato come descritto nella sezione 4 e contenente tutti i file prescritti, correttamente generati e/o compilati.

Alla consegna, eliminare tutti i dataset di prova nella cartella `data/`, escluso il dataset per i test automatici. Come già detto sopra, prima della consegna, occorre controllare che sia possibile scaricare tutto il progetto da Github come singolo file .zip. Se il progetto è troppo grande perché le interazioni con LLM sono numerose e con molti file, caricare su un utenza Drive (o equivalente) ad accesso libero lo zip del progetto, e inviare il link di accesso, con eventuali istruzioni, a docente e tutor con una mail.

Il progetto deve essere eseguibile automaticamente dal programma di valutazione usando i comandi standard descritti sopra, e le funzioni sviluppate come da specifica devono poter essere incluse ed eseguite. Ovviamente, il progetto deve anche essere eseguibile con intervento manuale usando la GUI sviluppata.

Le similitudini di tutti gli elaborati (programmi Python, file .yaml, file .md) saranno controllati con strumenti professionali per scoprire eventuali plagi. Si consiglia quindi di evitare che due o più gruppi lavorino insieme, e di curare l'originalità delle interazioni con LLM e della scrittura dei programmi Python.

In caso di problemi a eseguire i programmi del progetto, o per altre motivazioni, il gruppo potrà essere chiamato a presentare di persona o in call il progetto, o a effettuare correzioni.

Solo per i gruppi che consegnano entro il 30/6, che non hanno chiesto estensioni del tempo di consegna e il cui progetto non presenta problemi, saranno dati alcuni bonus, sommati al voto finale di Ingegneria del Software dei partecipanti al gruppo:

- +1 per il gruppo che trova la migliore (cioè minore) funzione costo QUBO sui dati di verifica del docente.
- +1 per i due gruppi con la migliore prestazione del classificatore sui dati di verifica del docente, secondo la metrica: F1 per target = 1.
- +1 per la migliore GUI (a giudizio insindacabile del docente).
- +1 per i migliori test automatici: non necessariamente i più numerosi (a giudizio insindacabile del docente).

Appendice – Costruzione della matrice QUBO per Feature Selection

Assumiamo di avere in partenza un dataset di rischio di credito, con i valori dei vari campi già convertiti a numeri (interi o reali) e normalizzati.

Il dataset può essere rappresentato come una matrice di features [U], le cui righe corrispondono agli *m* campioni e le cui colonne alle *n* features:

$$U = \begin{bmatrix} u_{11} & u_{12} & u_{13} & \dots & u_{1n} \\ u_{21} & v_{22} & u_{23} & \dots & u_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ u_{m1} & u_{m2} & u_{m3} & \dots & u_{mn} \end{bmatrix}$$

I valori target sono contenuti in un vettore V con *m* elementi, che corrispondono alle righe di medesimo indice:

$$V = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_m \end{bmatrix}.$$

Gli elementi di *V*, v_i, possono assumere solo i valori binari 0 (credito in bonis) o 1 (credito in sofferenza).

Dall'insieme originale di n caratteristiche vogliamo selezionare un sottoinsieme di K caratteristiche.

Matematicamente, il nostro obiettivo sarà trovare le colonne di U che sono correlate con V, ma non correlate tra loro. Supponiamo che il valore del coefficiente di correlazione possa assumere valori compresi tra -1 e 1. Sia ρ_{ij} la correlazione tra la colonna i e la colonna j della matrice U, e sia ρ_{Vj} la correlazione tra la colonna j di U e la singola colonna di V.

Per trovare il sottoinsieme "migliore", introduciamo n variabili binarie x_j, che hanno la proprietà:

$x_j = 1$, se la feature j è selezionata, $x_j = 0$ altrimenti.

Il vettore delle variabili binarie sia X:

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}$$

Associamo il sottoinsieme migliore di features al valore di X che minimizza una funzione obiettivo, che costruiamo a partire da due componenti. La prima componente rappresenta

l'influenza che le features hanno sulla classe da determinare, mostrata qui in una forma che aumenta all'aumentare del numero di termini:

$$\sum_{j=1}^{n} x_j |\rho_{Vj}|.$$

La seconda componente rappresenta l'indipendenza delle feature. L'espressione mostrata sotto aumenta all'aumentare del numero di termini correlati tra loro, che è l'opposto dell'indipendenza:

$$\sum_{j=1}^{n} \sum_{\substack{k=1, \\ k \neq j}}^{n} x_j x_k |\rho_{jk}|$$

Per ottenere una funzione obiettivo che sia massima nel punto ottimale, dovremo sottrarre questa formula dal primo termine. Combiniamo i due termini utilizzando un parametro α ($0 \leq \alpha \leq 1$) per rappresentare il peso che vogliamo assegnare all'indipendenza (massima per $\alpha = 0$) e all'influenza (massima per $\alpha = 1$). Quindi neghiamo l'espressione complessiva per ottimizzare al minimo, ottenendo:

$$f(\mathbf{x}) = -\left[\alpha \sum_{j=1}^{n} x_j |\rho_{Vj}| - (1-\alpha) \sum_{j=1}^{n} \sum_{\substack{k=1, \\ k \neq j}}^{n} x_j x_k |\rho_{jk}| \right].$$

Possiamo sfruttare la proprietà che $x_j = x_j\, x_j$ per le variabili binarie. Ciò significa che i termini $\alpha\, x_j\, |\rho_{Vj}|$ si possono mettere sulla diagonale della matrice QUBO Q, mentra i termini $-(1 - \alpha)\, |\rho_{jk}|$ sono gli elementi della riga j e colonna k di Q. Ciò ci consente di riscrivere la sommatoria come prodotto matriciale:

$$f(\mathbf{x}) = -\mathbf{x}^T Q \mathbf{x}\,.$$

Da questa formula, possiamo esprimere il problema in termini dell'operatore argmin, che restituisce il vettore x^* per il quale il suo argomento (una funzione) è minimizzato:

$$\mathbf{x^*} = \underset{\mathbf{x}}{\operatorname{argmin}} \left[-\mathbf{x}^T Q \mathbf{x} \right].$$

Il problema di minimizzazione è un classico problema QUBO, che sappiamo essere *np-hard*.

Le correlazioni tra le colonne di U (le singole features per tutti i campioni) e il vettore target V, ρ_{Vj}, e tra le coppie di colonne, ρ_{ij}, è calcolata usando la formula di Spearman, che è più adatta rispetto alla classica correlazione di Pearson, che assume la linearità.

Nota Q, la minimizzazione si esegue usando uno dei tanti algoritmi open source di risoluzione QUBO reperibili online. Ovviamente, non tutti gli algoritmi sono uguali, e va riposta cura per scegliere e usare il migliore, o uno dei migliori per il problema in oggetto. Si noti che la matrice Q del nostro problema è densa, cioé praticamente tutti i suoi elementi sono diversi da zero.

Una volta trovato il vettore X che contiene l'indicazione delle feature ottime da utilizzare per la classificazione, la matrice U si riduce eliminandone le colonne che corrispondono ai valori $x_j = 0$, e si utilizza la matrice U' ridotta per la classificazione.

Si noti che il risultato dell'ottimizzazione QUBO dipende dal valore di α. Per valori minori di α si troveranno meno colonne di U, mentre per α = 1 l'ottimizzazione restituirà necessariamente tutte le colonne (U' = U).

Il sistema deve quindi effettuare varie prove al variare di α, per arrivare al numero dato di features da eliminare.